iMovies

MOVIE TICKET BOOKING SYSTEM

Full Stack Development with MERN

Project Documentation

Student	Nidhi Rathore (Individual Project)
Institute	Medi-Caps University
Department	Computer Science and Engineering (CSE)
Course	Full Stack Development – MERN Stack
Submission Date	22 October 2026

Prepared for academic project submission

1. Introduction

Project Title: iMovies – Movie Ticket Booking System

iMovies is a full-stack web application for discovering movies, viewing showtimes, selecting seats and managing movie-ticket bookings. It connects a React frontend with a Node.js/Express backend and a MongoDB database. The project demonstrates common full-stack workflows, authentication, booking management and role-based administration.

Student: Nidhi Rathore (Individual Project)

Institute: Medi-Caps University | **Department:** Computer Science and Engineering (CSE)

2. Overview Project

Purpose

The purpose of iMovies is to provide one web interface for browsing movies, viewing showtimes, selecting seats, completing a demo checkout and managing bookings.

Main Features

- User registration, login and logout.
- Browse currently showing and upcoming movies; search/filter and open movie details.
- View theatres and showtimes, select seats and place temporary seat holds.
- Simulated payment success/failure flow; view booking details and booking history.
- Eligible booking cancellation and demo-refund flow.
- Admin functions for movies, theatres, screens, shows, bookings and registered users.

3. Architecture

Frontend: React 18 with Vite, JavaScript, Bootstrap 5, React Router and Axios.

Backend: Node.js 18+ and Express.js REST API.

Database: MongoDB, accessed through Mongoose.

Authentication and security: JWT stored in an HTTP-only cookie, password hashing, request validation, Helmet, rate limiting and admin-role checks.

4. Setup Instructions

Prerequisites

- Node.js 18 or later and npm.
- Git.
- A reachable MongoDB database, such as MongoDB Atlas.

Installation

Clone the repository, then install dependencies separately in the client and server directories.

```
git clone https://github.com/rnidhi4/iMovies.git
cd iMovies

cd client
npm install

cd ../server
npm install
```

Environment variables

In the server directory, create a local `.env` file from `.env.example` and set the required values. The README lists `MONGODB_URI`, `JWT_SECRET` and `CLIENT_URL`. The frontend can optionally use `VITE_API_URL`. Never commit secrets or a real `.env` file.

Note: Keep actual credentials private. The required values depend on the local/deployed environment configuration.

5. Folder Structure

The repository is organized into a React client, an Express server and supporting documentation.

```
iMovies/
  client/src/ — api, components, context, hooks, layouts, pages, routes, utils
  server/src/ — config, controllers, middleware, models, routes, services, utils, validators
  server/scripts/ — backend utility scripts
  docs/ and Phase wise Doc/ — project documentation and testing material
  README.md — setup and project overview
```

6. Running the Application

Run the backend and frontend in separate terminals.

Backend terminal

```
cd server
npm run dev
```

Frontend terminal

```
cd client
npm run dev
```

The README documents the usual local addresses as backend http://localhost:5000 and Vite frontend http://localhost:5173. Health check: /api/v1/health. Local ports may differ if configured differently.

7. API Documentation

The backend API uses the base prefix `/api/v1`. The following endpoint overview is based on the supplied documentation and the repository README. Request bodies, query parameters, response examples and status codes are not reproduced where they were not fully specified in those sources.

Area	Method and endpoint	Purpose
Health	GET /health	Check backend health.
Authentication	POST /auth/register POST /auth/login POST /auth/logout GET /auth/me	Register, sign in/out and retrieve the current user session.
Movies	GET /movies GET /movies/:id GET /movies/:id/shows	List movies, view a movie and retrieve its shows.
Theatres / Shows	GET /theatres GET /shows/:id GET /shows/:id/seats	Browse theatres, show details and seat availability.
Bookings	GET /bookings POST /bookings/hold GET /bookings/:id POST /bookings/:id/cancel	List bookings, hold seats, view a booking and cancel when eligible.
Demo payment	POST /payments/demo/:bookingId	Run the simulated payment flow; no real payment is processed.
Admin	GET /admin/stats GET /admin/users GET /admin/bookings POST /admin/bookings/:id/cancel	Dashboard stats, user/booking access and admin cancellation.
Admin management	POST/PUT/DELETE /admin/movies GET/POST/PUT/DELETE /admin/theatres POST/DELETE /admin/screens GET/POST/PUT /admin/shows	Manage movies, theatres, screens and shows. Shows are not deletable through the admin interface according to the README.

Example health-check request: `GET /api/v1/health`. A successful response format and status code are not asserted here because the existing documentation does not specify an exact response body.

8. Authentication

Users register and log in through the authentication API. Passwords are hashed, and the JWT is stored in an HTTP-only cookie. Protected routes require authentication, admin operations require an admin role, and logout clears the authentication cookie. The application uses a simulated payment flow; it does not process real payments or collect card details.

9. User Interface

The supplied screenshots show the iMovies interface with a dark theme, movie navigation, search/filter controls, ratings, movie cards and a featured movie details panel. These are genuine screenshots supplied for this documentation.

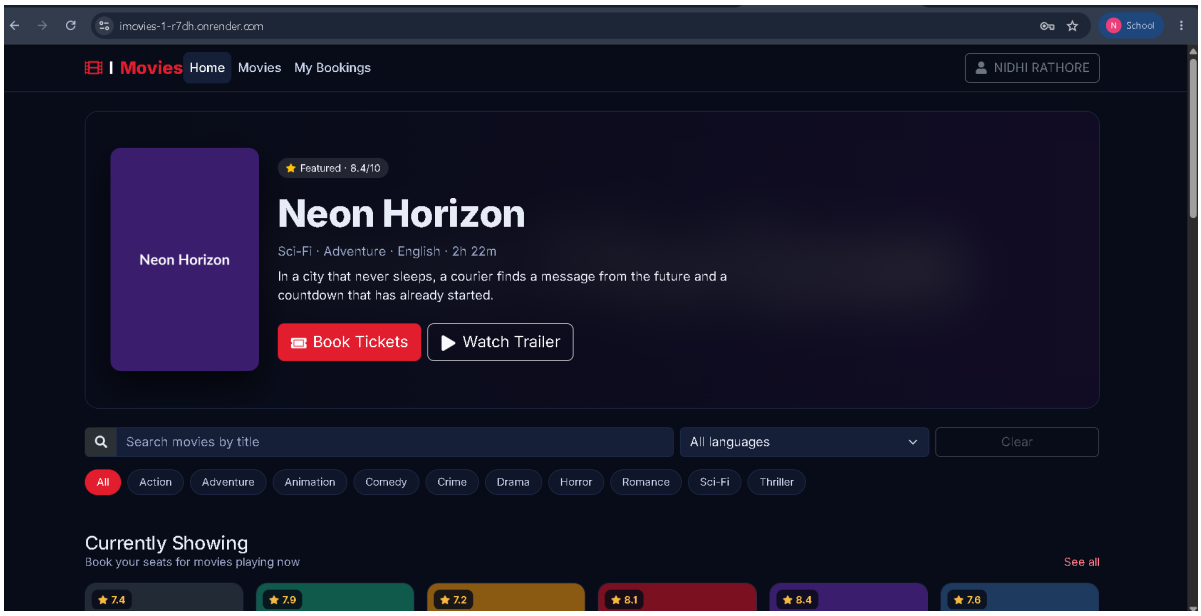


Figure 1. Home page with featured movie, search, language filter and genre controls.

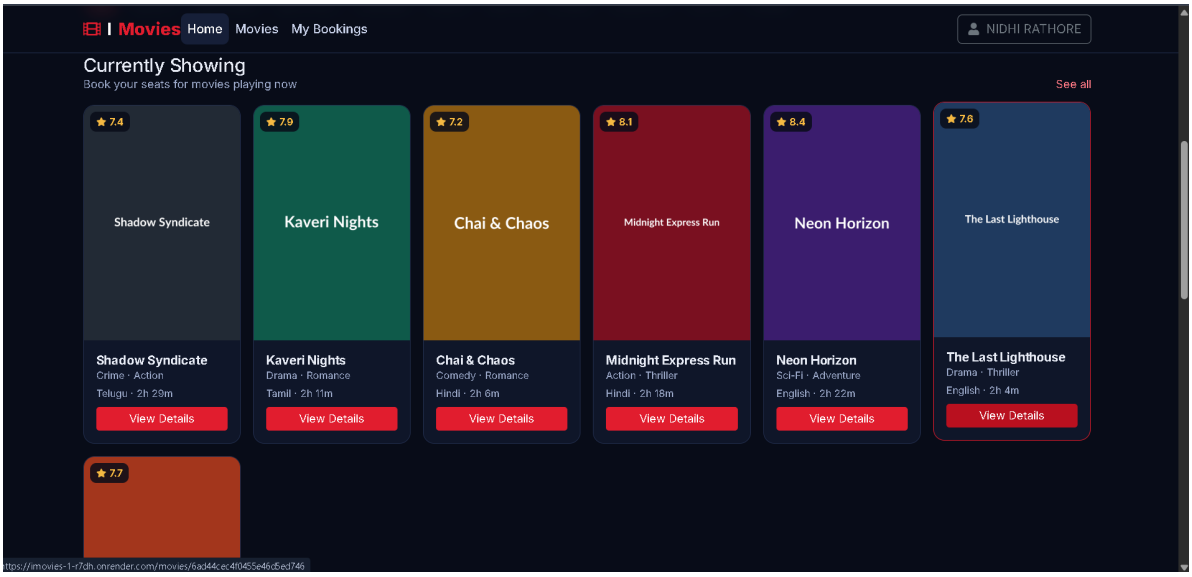


Figure 2. Currently showing movie cards with ratings, genres, languages and details buttons.

10. Testing

The existing documentation refers to functional test cases in `Phase wise Doc/Performance Testing/02-Functional-Test-Cases.md`. Suggested areas include registration, login/logout, movie browsing, showtimes, seat holds, demo payment success/failure, booking history/cancellation and admin authorization.

Test results: No pass/fail result is claimed because observed execution results were not provided. Automated testing and performance measurements are not claimed as completed.

---

11. Screenshots or Demo

The following genuine screenshot, supplied for this documentation, shows the upcoming-movies section.

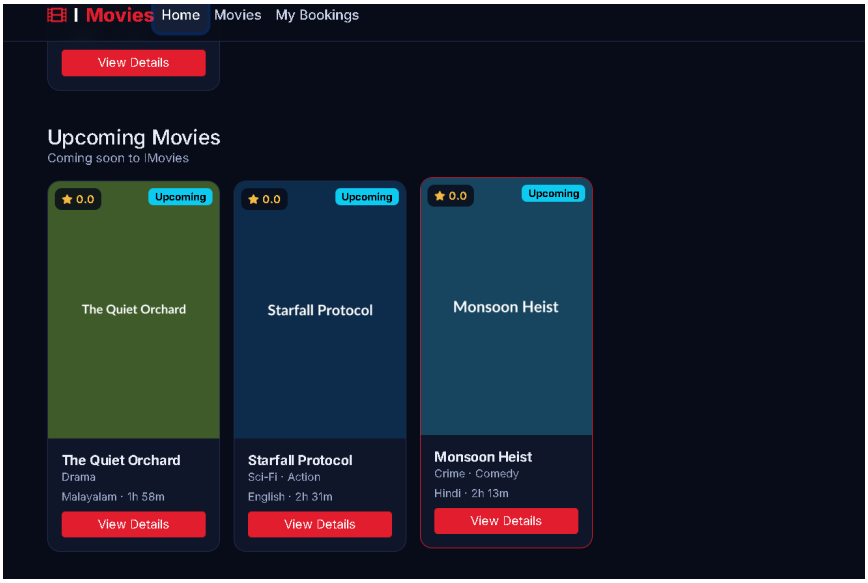


Figure 3. Upcoming movies section with status labels and movie details buttons.

GitHub repository: https://github.com/rnidhi4/iMovies

Live demo: The supplied screenshot shows `imovies-1-r7dh.onrender.com`, but availability could not be confirmed during preparation. It is not presented as a verified working demo link.

12. Known Issues

The following limitations are documented in the existing project material:

- Payments and refunds are simulated; there is no real payment gateway.
- Email/SMS notifications and QR-code tickets are not included.
- Shows cannot be deleted through the admin interface.
- Customer show listings are limited to the upcoming seven-day window.
- Performance measurements have not been recorded in the supplied documentation.

13. Future Enhancements

Possible improvements that fit the current project scope include:

- Integrate a real payment provider in sandbox mode before any production payment use.
- Add email confirmations, QR-code tickets and downloadable invoices.
- Add movie reviews and a watchlist.
- Expand functional, automated and concurrent-seat-booking tests.
- Measure and optimise API response times and page performance.
- Improve accessibility and mobile responsiveness.

Source and accuracy note

This document reuses the supplied iMovies documentation and project screenshots, with the student/institute details provided for this submission. Repository setup and feature descriptions were cross-checked against the public GitHub README. Where the source material did not establish a detail—such as exact response bodies, actual test outcomes or a currently reachable demo—the report does not invent it.

Repository: github.com/rnidhi4/iMovies